

OPTIMIZING THE WORKING SCHEDULE FOR FACTORY WORKERS

Based on the data from the statement and from the Internet, we constructed a mathematical model for the staff scheduling problem for a dairy factory.

We first made some assumptions about the working condition. Then, we constructed the decision variables, the constraints, and the objective function of the model. From that, we created an implementation, and use it to construct various working schedules for each dataset and other factors.

Next, we analyzed the outputted plans along with the model itself. Finally, we decided on the most effective strategy for each dataset.

Contents

1	Introduction	3
2	Assumptions	4
3	Mathematical model	5
3.1	Variables	5
3.2	Constraints	5
3.3	Dissatisfaction	6
3.4	Fairness	6
3.5	Objective function	7
3.5.1	Subtask A	7
3.5.2	Subtask B	7
4	Implementation & Execution of the model	9
4.1	Subtask A	9
4.1.1	Algorithm 1	9
4.1.2	Algorithm 2	10
4.2	Subtask B	12
4.2.1	Dataset 1	12
4.2.2	Dataset 2	12
5	Analysis of the model	14
5.1	Subtask A	14
5.2	Subtask B	14
5.3	Overall	14
6	Conclusion	16

1 Introduction

The staff scheduling problem is one encountered by many, if not all manufacturing companies. To function properly, factories must plan each employee's schedule in such a way that satisfies various constraints while minimizing resources and risks. With current technologies, this type of problem can be modeled mathematically and automated.

The number of constraints in a staff scheduling problem, in some cases, can become very complicated, and even seemingly contradictory. Local laws and regulations, for example, introduce constraints on the maximum number of working hours per month, the minimum salary and wage, etc. Other constraints include each worker's skillset, which limits positions that they can work on, or their previous shifts, since contracts may prohibit schedules that assign multiple shifts in a short amount of time. Without a mathematical model, these schedules must be constructed manually, and most of the time, simplified with methods such as grouping employees, introducing unnecessary redundancy.

The optimization target is also a matter of major concern. While it may seem trivial that generated profit is the sole target, sacrificing other factors may produce detrimental effects. For example, generating an unfair schedule (which still satisfies all hard constraints but assigns work unevenly) can result in a strike, introducing further profit loss. A mathematical model can take into consideration these risk factors to optimize expected profit.

Furthermore, modeling a factory's work schedule also gives various insights into the factory's overall structure. With these insights, it is possible to generate lay-off plans while still guaranteeing all constraints.

In this paper, we discuss methods to mathematically model the staff scheduling problem for a dairy factory.

2 Assumptions

In reality, there are various uncontrollable factors, including both technical issues and human resources issues. To avoid overcomplication, we make the following assumptions:

- Every worker perform follows the generated schedule, as long as the schedule satisfies every constraint in the problem statement.
- Every machine performs perfectly as stated in the problem statement.

Additionally, we assume other things relating to the working condition:

- The time needed for changing shifts is negligible.
- We assume that the dissatisfaction of each job is identical. This means that it does not matter for workers what job they have to do, as long as they are capable of doing so.

3 Mathematical model

We redefine the schedule to have $28 \times 3 = 84$ shifts, numbered from $0 \rightarrow 83$. The shifts with index $\equiv 2 \pmod{3}$ are night shifts.

3.1 Variables

We have the following variables:

$X = \{x_{ijkl}\}$: let x_{ijkl} be a binary variable denoting that worker i work in shift j in pipeline k and performing job l or not.

$A = \{a_{ikl}\}$: let a_{ikl} be a binary variable denoting that worker i is capable of doing job l in pipeline k or not.

$R = \{r_{kl}\}$: let r_{kl} be the number of workers that are needed for job l in pipeline k to perform normally.

$T = \{t_{jk}\}$: let t_{jk} be the number of demanded hours in shift j for pipeline k .

(We decomposed the demand periods into smaller ones so that each small period lies completely inside a shift^[1])

3.2 Constraints

1. Every person only does the jobs they are capable of:

$$x_{ijkl} \leq a_{ikl}$$

2. After performing the night shift, workers are not allowed to work the morning shift of the next day:

$$\sum_{j \equiv 2 \pmod{3}, k, l} (x_{ijkl} + x_{i(j+1)kl}) \leq 1$$

3. Every day, a worker is only permitted to work one shift:

$$\sum_{j:3,k,l} (x_{ijkl} + x_{i(j+1)kl} + x_{i(j+2)kl}) \leq 1$$

4. For every shift j and pipeline k , there have to be enough workers to perform the job:

$$r_{kl} \leq \sum_i x_{ijkl} \quad \forall j, k, l, t_{jk} > 0$$

5. In subtask B, a single worker can only work in a maximum of 24 days:

$$\sum_{j,k,l} x_{ijkl} \leq 24$$

3.3 Dissatisfaction

We have a formula for calculating the value of dissatisfaction:

U_j is the dissatisfaction coefficient for the worker working in shift j in an hour.

We have the following table for the coefficients:

	Monday to Friday	Saturday	Sunday
Morning and afternoon shift	1	1.1	1.7
Night shift	1.5	1.65	1.87

3.4 Fairness

We have a formula for calculating the fairness for workers^[2]:

Let S_i be the sum of dissatisfaction for worker i .

$$S_i = \sum_{j,k,l} U_j * t_{jk} * x_{ijkl}$$

Let F be the fairness of the workers.

$$F = \max S - \min S$$

Let Δ be the maximum difference between S_i and the average value of S (which is $\bar{S}$).

$$\Delta = \max |S_i - \bar{S}|$$

3.5 Objective function

3.5.1 Subtask A

Since optimizing the number of workers in each shift is the top priority, we give a very large weight to it in the objective function. Furthermore, the objective function also has to take into account the fairness between workers. Thus, we suggest the following objective function:

$$OP = c_n * N + c_f * F$$

In which:

- OP is the objective value of the model. The smaller OP is, the more optimized the solution is.
- c_n is the weight for optimizing the number of workers in each shift.
- N is the total number of shifts of all workers.
- c_f is the weight for optimizing the fairness of workers.
- F is the fairness of the model. The smaller F is, the more balanced the model is.

3.5.2 Subtask B

Subtask B requires us to minimize the total cost. We believe the formula for total cost should contain a constant salary for each worker, along with an hourly wage for every working hour:

$$OP = C_s * \sum_i [\sum x_{ijkl} \geq 1] + C_h * \sum x_{ijkl}$$

In which:

- OP is the objective value of the model. The smaller OP is, the more optimized the solution is.

- C_s is the constant salary of each worker. We will only calculate the constant salary for workers that are hired.
- C_h is the hourly wage for each worker.

4 Implementation & Execution of the model

The results of the model can be found below^[3]. The results for each subtask are in Paper_PartA and Paper_PartB respectively.

Based on real-life data, we set the constant salary to be 1,600,000 VND, and the hourly wage to be 10,000 VND per hour. However, these number are scalable.

4.1 Subtask A

We discuss 2 different algorithms in order to minimize the objective function. As the two different optimization targets in subtask A are not directly related, our algorithms try to impose additional constraints and introduce additional variables, in order to transform a non-linear model into a linear model so that it can be solved efficiently.

Inside Paper_PartA, results with *maindelta* is for algorithm 1, and *main* is for algorithm 2.

4.1.1 Algorithm 1

In the first algorithm, we minimize the number of workers using a constraint, and then try to minimize the fairness of the model. This is because the number of required workers for each shift is constant and can be calculated.

We will introduce additional constraints: The number of workers in each shift that has a demanded period, doing a specific job in a specific pipeline must be equal to R .

We can see that Δ and F are directly related to each other ($F \leq \Delta \times 2$), thus minimizing Δ leads to smaller F . Thus, we can try to keep all S_i in the range $[\bar{S} - \Delta, \bar{S} + \Delta]$. We then try to minimize Δ according to the above constraints.

For the first dataset, we run this model three times, each with a different planning horizon. Below is the data for these three runs:

Due to the nature of the model, the number of required workers for each job and shift is constant. As such, we have three constants, $\bar{S} = 211.515$,

$\sum S = 3595.75$, and the total cost is 48,400,000 VND.

Horizon	$\min S$	$\max S$	$\max S - \min S$	Δ
1 day	168.3	248.1	79.8	43.2147
3 days	167.5	244.6	77.1	44.0147
7 days	179.65	243.3	63.65	31.8647

Overall, it can be seen that $\max S - \min S$ decreases as the horizon length is increased. We believe that this is because the more time slots the model gets to schedule, the easier it is to balance employees' dissatisfaction.

However, quite unintuitively, the value of Δ does not correlate with the horizon length. As the running time of the model increases dramatically as the horizon length increases, we were not able to perform more tests. We believe that these two variables will correlate negatively with a larger sample size. Note that in this model, we only use Δ as a means of approximating $\max S - \min S$.

For the second dataset, due to limitations in our computing power, we only ran this model once.

Similarly, for the second dataset, the constants are $\bar{S} = 111.869$, $\sum S = 6152.8$, and the total cost is 114,400,000 VND.

Horizon	$\min S$	$\max S$	$\max S - \min S$	Δ
1 day	44.3	234.55	190.25	122.681

In conclusion, while this model is capable of yielding results optimising fairness to the minimum, it's running time is not desirable. Based on results for the first dataset, increasing the horizon length does not yield significantly better fairness.

4.1.2 Algorithm 2

In the second algorithm, instead of minimizing the fairness of the model, we set the fairness range as the constraint, and then find a solution for the model that does not introduce

Similar to algorithm 1, we will try to keep S_i in the range $[\bar{S} - \Delta, \bar{S} + \Delta]$. However, we will set this as a constraint, then manually minimize Δ and

optimize the number of workers instead.

Our implementation of this algorithm calculates Δ after every iteration, and then tries to reduce Δ by multiplying it with a small constant (for instance, 0.9). If the new model is infeasible to solve, or if the number of workers required over all shifts increased by too much (which occurs when Δ is too small), then we will stop this process.

This optimization is performed after running every shift, in order to incorporate the new schedules. That is, we run the model without Δ constraints once, then starts to reduce Δ .

Below is the dataset for our runs on the first dataset.

Similar to the first algorithm, the number of required workers for each job and shift is constant. As such, we have three constants, $\bar{S} = 211.515$, $\sum S = 3595.75$, and the total cost is 48,400,000 VND.

Horizon	$\min S$	$\max S$	$\max S - \min S$	Δ
1 day	172.8	244.2	71.4	38.71
3 days	174	244.7	70.7	37.51
7 days	169.35	243.95	74.6	42.16
28 days	178.6	246.4	67.8	34.88

For this model, we were able to run 2 times on the second dataset. Below is the data:

Once again, constants for the second dataset are $\bar{S} = 111.869$, $\sum S = 6152.8$ and the total cost is 114,400,000 VND.

Horizon	$\min S$	$\max S$	$\max S - \min S$	Δ
1 day	45.4	245.95	200.55	134.08
3 days	47.8	222.9	181.4	117.33

Regarding the running time, let Δ' be the target value for Δ , we perform $O(\log \frac{\Delta}{\Delta'})$ iterations to optimize Δ . In reality, this translates to approximately 3 – 10 model optimization calls. However, due to the structure of this model, the running time is significantly better.

In conclusion, compared to the first algorithm, this second algorithm yields better result, since Δ is optimized using heuristics, rather than using

integer programming directly. Overall, this model performs better than the first model in both results and running time.

4.2 Subtask B

We create a single integer programming (IP) algorithm, which takes X as decision variables and minimizes the objective function.

4.2.1 Dataset 1

First, we run the program with the default horizon (1 day ahead), and we have our result in `result_data_1_part_b_1_days.txt`.

We can see that in each shift, there are $1 + 2 + 2 = 5$ people that are needed for the factory to function properly. Moreover, there are 3 shifts each day (for instance, day 1 demands all 3 shifts), and each person can only work in 1 shift every day. Thus, we need at least 15 different people.

The program has given a plan which utilizes exactly 15 people, and each shift has reached the minimal number of workers allowed. For that reason, the program has found an optimal solution, while only knowing the schedule for the next day.

In order to verify that, we ran the program where we know the entire schedule beforehand (setting the horizon to 28 days), and the result is stored in `result_data_1_part_b_28_days.txt`. As expected, although the program output a different plan, the total cost remained the same, which is 45,200,000 VND.

4.2.2 Dataset 2

Similarly, we ran the program with different horizon factors (1 and 28 days respectively), and the results are stored in `result_data_2_part_b_1_days.txt` and `result_data_2_part_b_28_days.txt`.

The program did output a valid solution if it knows the entire schedule, but it fails to do so when only the schedule for 1 day is given. Only when we run the model for a 3-day horizon did the model give a valid plan in `result_data_2_part_b_3_days.txt`.

We also come to the same conclusion as for the first dataset: The plan for 3-day horizon and 28-day horizon have the same optimal total cost, which is 108,000,000 VND.

5 Analysis of the model

5.1 Subtask A

The most detrimental weakness of the model for subtask A is that it cannot run for large values planning horizon.

When the number of days that we can see ahead increases, there are too many constraints, which make the model unable to be solved by popular linear programming solvers (in this case, Gurobi).

For algorithm 1, it is impossible to run with a 28-day horizon for dataset 1, and it is only possible to run with a 1-day planning horizon for dataset 2; for algorithm 2, it is not possible to run a 7-day horizon for dataset 2.

5.2 Subtask B

While we were running the program for dataset 2, we recognize the following problem:

If the constant salary is greater than 0, the program will try to reuse the workers that have been hired before. However, each worker can only work for 24 days, which means that after this, they cannot accept any more shifts.

Then, the program will have to hire new workers. However, the remaining workers are not enough, since every worker can only work only 1 shift every day.

So, there are 2 possible solutions to this issue:

- **Reducing the constant salary to 0:** This will help the program avoid reusing those workers, avoiding the above problem.
- **Increasing the planning horizon:** Extending the planning horizon will help the program let those who have worked a lot have rest days in order to resolve the matter.

5.3 Overall

The given model utilized hard constraints, such as "every person can only work in 24 days", or "a shift has to be performed even if only a single hour

is in the demand period”.

This is a weakness of the model because realistically, a worker can work overtime for 1 hour; similarly, that person can accept to work an extra day (for a total of 25 days). As such, the model to not be the exact representation of real-world scenarios.

6 Conclusion

For subtask A, our model has successfully created a plan to minimize the fairness with horizon 1. With the larger horizon, our model spent a lot of time calculating the result.

For subtask B, dataset 1, our model has successfully created a plan to minimize the total cost with any planning horizon. However, for dataset 2, our model was unable to find a way to calculate the cost with horizon 1 because of the characteristics of the model.

References

- [1] B. Jaumard, F. Semet, and T. Vovor, “A generalized linear programming model for nurse scheduling,” *European Journal of Operational Research*, pp. 1–18, 1998.
- [2] I. Blöchliger, “Modeling staff scheduling problems. a tutorial,” *European Journal of Operational Research* 158, pp. 533–542, 2004.
- [3] “Results of the model.” [Online]. Available: https://drive.google.com/drive/folders/17mN4bqVQaNUu5_pbADDFuG9H6iisjSqn